# VENUEVISTA

## Smart Venue Management & Event Reservation Platform with Host Approval Lifecycle & Business Hub

**Project Title:** VenueVista Full-Stack Web Platform

**Author / Developer:** Rudra Srivastava

**Technology Stack:** Spring Boot 4 (Java 21) • Angular 21 (TypeScript) • Spring Data JPA • H2/MySQL • JWT

**Repository:** github.com/srivastavarudra22-prog/venue-vista

**Document Type:** Project Submission & Interface Report

**Submission Date:** October 2026

| | | | |
|---|---|---|---|
| **Status:** Fully Functional | **Architecture:** 3-Tier REST | **Frontend:** Port 4200 | **Backend:** Port 8080 |

# 1. Executive Summary & Problem Analysis

**VenueVista** is an enterprise-grade, full-stack event and venue management platform engineered to resolve the disorganization and friction prevalent in traditional venue booking. By bridging the gap between consumers looking for event spaces and venue managers hosting properties, VenueVista delivers a unified marketplace featuring transparent booking requests, real-time approval/rejection decision flows, and synchronized event ticketing.

## The Problem in Traditional Venue Booking:

- **Fragmented Venue Discovery:** Customers typically struggle across disparate phone contacts, classifieds, and social media without transparent capacity, amenities, or pricing information.
- **Schedule Clashes & Double Bookings:** Manual booking records without real-time date verification lead to calendar conflicts, embarrassing double bookings, and dissatisfied clients.
- **Absence of Host Control:** Venue providers lack a modern management portal to view incoming customer requests, review dates, and accept or reject reservations with clear rationale.
- **Opaque Rejection Feedback:** In legacy systems, customers whose requests are rejected receive no explanation regarding whether the venue was booked offline, undergoing maintenance, or over capacity.
- **Isolated Ticketing Ecosystems:** Venue management and event ticket reservation systems typically exist as disconnected applications rather than an integrated platform.

## The VenueVista Solution:

- **Dual-Portal Architecture:** Distinct interfaces for Customers (booking venues and purchasing tickets) and Venue Hosts/Providers (managing listings and approving requests).
- **Request-Based Lifecycle:** Venue bookings initiate in a PENDING status, preventing uncoordinated confirmations until approved by the host.
- **Mandatory Rejection Reasons:** If a host rejects a request, they provide a specific reason (selected via templates or custom written) which is displayed directly on the customer's ledger.
- **Real-Time Financial Metrics:** Hosts track confirmed revenue, pending requests, total bookings, and active venues via real-time KPIs.
- **Zero-Friction Zero-Config Database:** Runs immediately out of the box with embedded H2 (MySQL compatibility) and auto-seeds initial venues, events, and user accounts.

# 2. System Architecture & Technical Specifications

VenueVista employs a clean **3-Tier Decoupled Client-Server Architecture**. The presentation tier is developed with **Angular 21**, utilizing standalone components and reactive HTTP services. The business and API layer is built on **Spring Boot 4 (Java 21)** with layered Controller-Service-Repository architecture, stateless JWT authentication, and Spring Data JPA for object-relational mapping.

## Technology Stack Matrix:

| Layer | Technology / Tool | Purpose & Details |
|---|---|---|
| Frontend Client | Angular 21, TypeScript 5.9 | Standalone architecture, reactive forms, client routing |
| Styling / UI | Modern CSS3, Flexbox/Grid | Responsive card UI, status badges, interactive modals |
| Backend Framework | Spring Boot 4, Java 21 LTS | RESTful controllers, service layer, dependency injection |
| Persistence & ORM | Spring Data JPA / Hibernate | Automated DDL, JPQL queries, relational entity mapping |
| Database | Embedded H2 / MySQL 8.x | Default zero-config H2 file store with MySQL dialect |
| Security & Auth | JWT (JSON Web Tokens), BCrypt | Stateless authentication, hashed passwords, route guards |
| Build Tools | Maven (mvnw), npm | Multi-module build, dependency management, CI/CD |

## Core Database Entities & Schema:

| Entity | Primary Fields | Relationships & Role |
|---|---|---|
| User | id, name, email, password, phone, role | Represents Customers, Providers, and Admins |
| Venue | id, name, location, capacity, pricePerDay, available | Property listings hosted by providers |
| Booking | id, eventDate, guests, totalAmount, status, rejectionReason | Links User & Venue with approval state lifecycle |
| Event | id, name, eventDate, ticketPrice, totalSeats | Scheduled events hosted at designated venues |
| Ticket | id, event_id, seat_id, price, status | Customer ticket reservations with seat numbers |
| Payment | id, booking_id, amount, paymentMethod, paymentStatus | Records settlement and checkout transactions |

# 3. REST API Specifications & Approval Lifecycle

## RESTful Endpoints Overview:

| Method | Endpoint | Role | Functionality |
|---|---|---|---|
| POST | /api/users/register | Public | Registers customer or provider account |
| POST | /api/users/login | Public | Authenticates credentials, returns signed JWT |
| GET | /api/venues | User | Retrieves all venues with capacity and price |
| POST | /api/venues | Provider | Creates a new venue inventory listing |
| DELETE | /api/venues/{id} | Provider | Deletes a venue from inventory |
| POST | /api/bookings | Customer | Submits reservation request with status PENDING |
| GET | /api/bookings | Provider | Lists all booking reservations for review |
| PUT | /api/bookings/{id}/status | Provider | Updates status (CONFIRMED / REJECTED with reason) |
| GET | /api/events | User | Lists upcoming events and seat capacity |
| POST | /api/payments | Customer | Records payment settlement for reservation |

## The Step-by-Step Approval & Rejection Workflow:

**Step 1 (Customer Request):** Customer selects a venue, specifies event date, and guest count. The booking record is stored with status PENDING.

**Step 2 (Host Alerting):** The reservation immediately appears on the Provider Hub under 'Pending Review', alerting the host to customer details (name, email, phone, guests).

**Step 3A (Host Approval):** Host clicks ✓ Approve. The backend updates status to CONFIRMED. The customer's ledger shows '✓ Approved by Host'.

**Step 3B (Host Rejection with Reason):** Host clicks ✕ Reject. An interactive modal opens prompting the host for an explanation (e.g. maintenance, double booking, capacity limits).

**Step 4 (Feedback Display):** The rejection explanation is stored in the database and rendered immediately in an alert box on the customer's 'My Bookings' screen.

# 4. Application Interface Walkthrough

**Figure 1: Customer Login Portal (/login)**

*Features responsive input validation, error handling, and a direct shortcut switch to the Venue Provider Portal.*

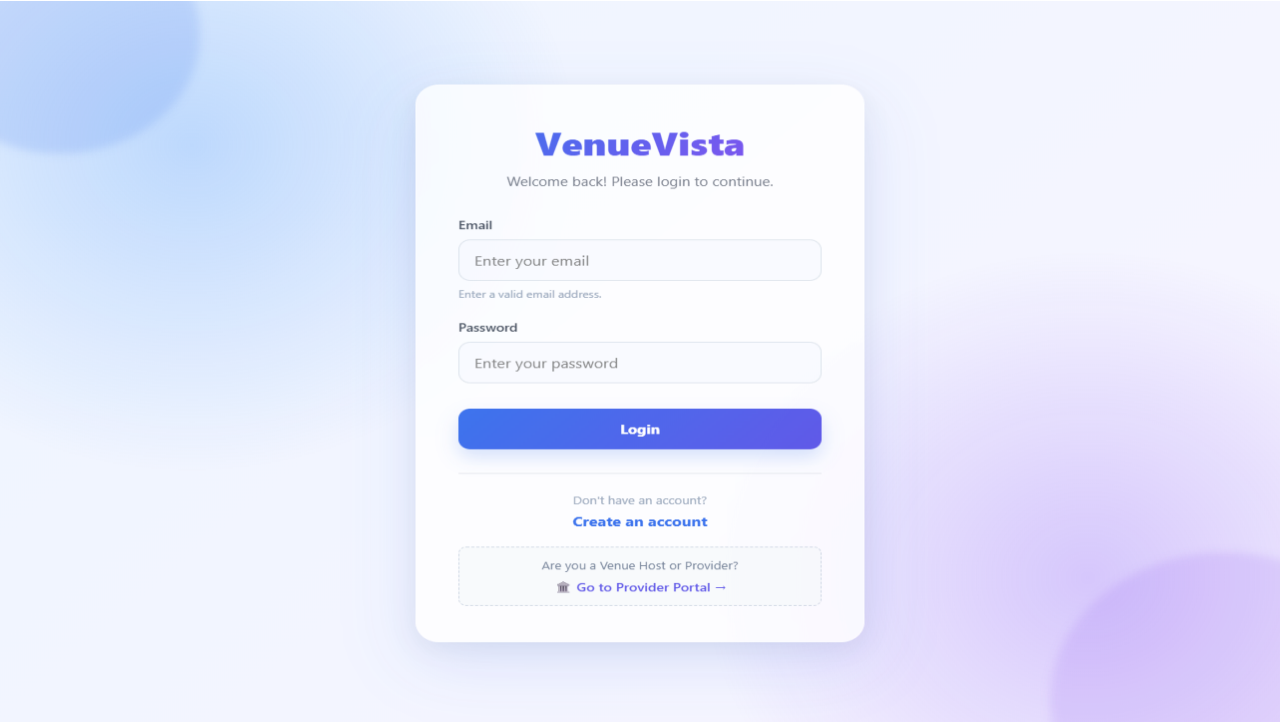
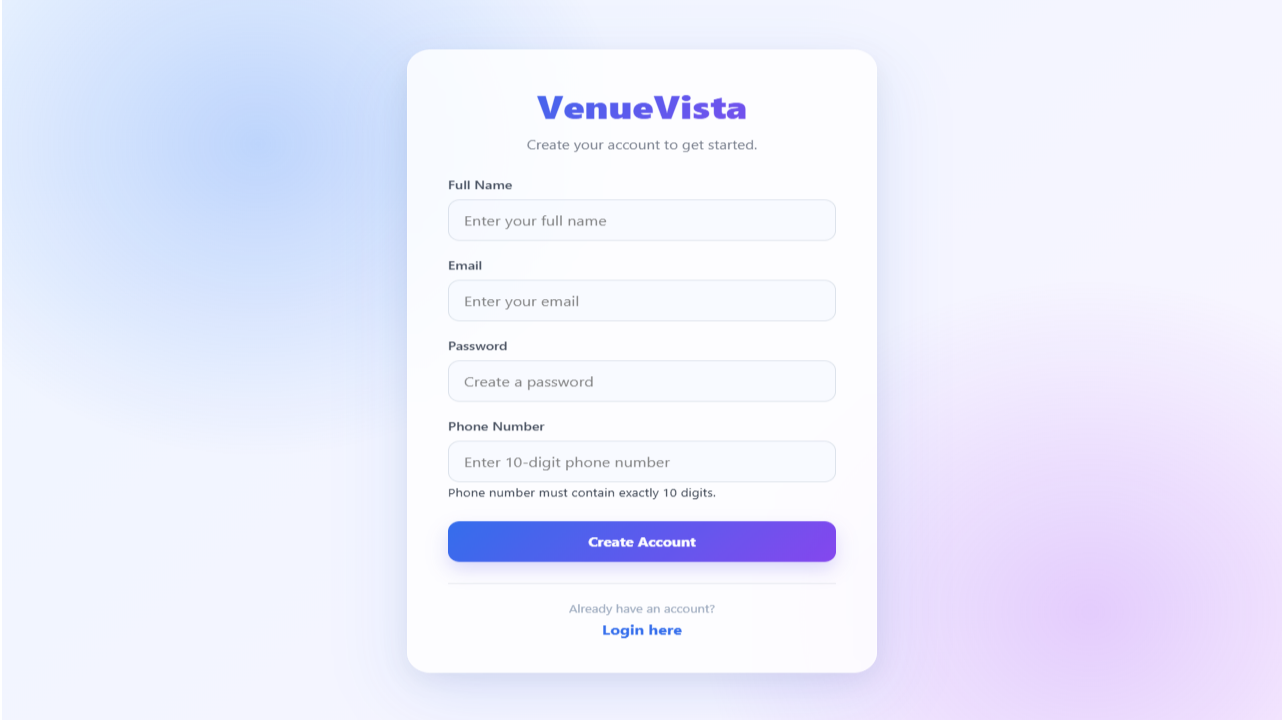



**Figure 2: Customer Account Registration (/register)**

*Captures customer name, email, encrypted password, and 10-digit mobile number with real-time field validation.*


VenueVista
Create your account to get started.
Full Name
Enter your full name
Email
Enter your email
Password
Create a password
Phone Number
Enter 10-digit phone number
Phone number must contain exactly 10 digits.
Create Account
Already have an account?
Login here

# Customer Discovery & Booking Experience

**Figure 3: Customer Main Dashboard (/dashboard)**

*Central navigation hub offering access to Venues, Upcoming Events, My Bookings, and Purchased Tickets.*

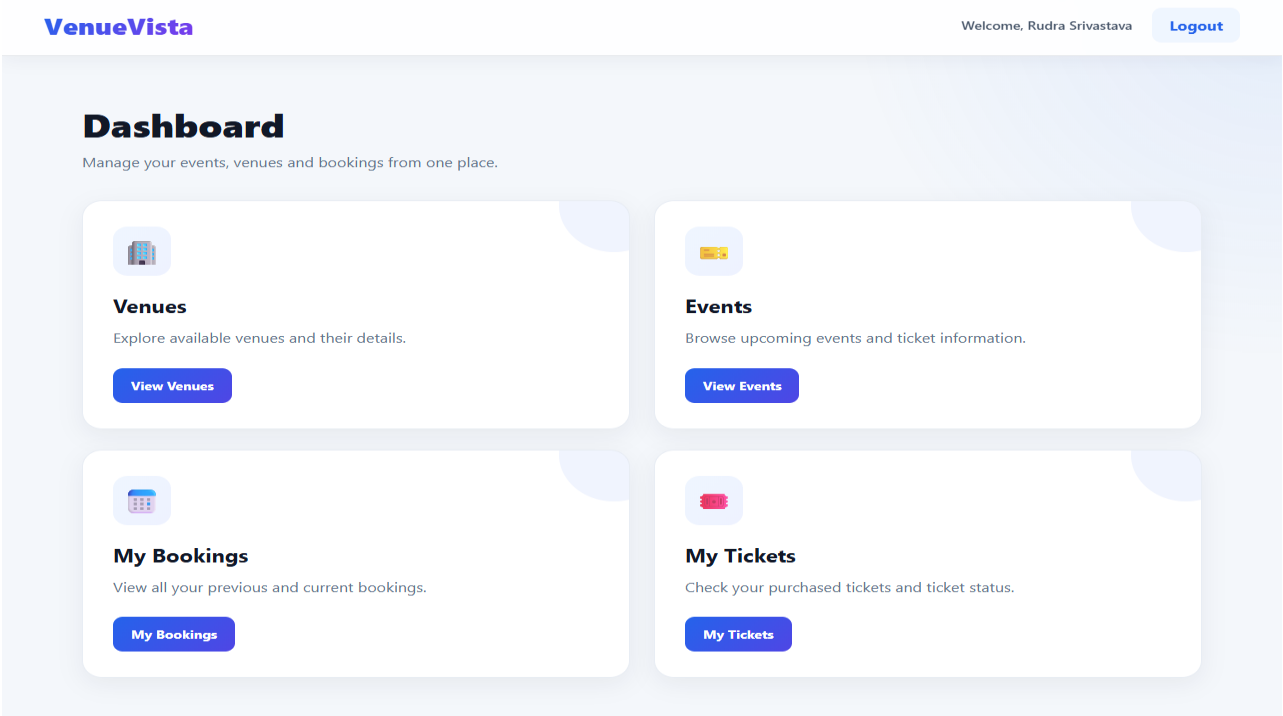
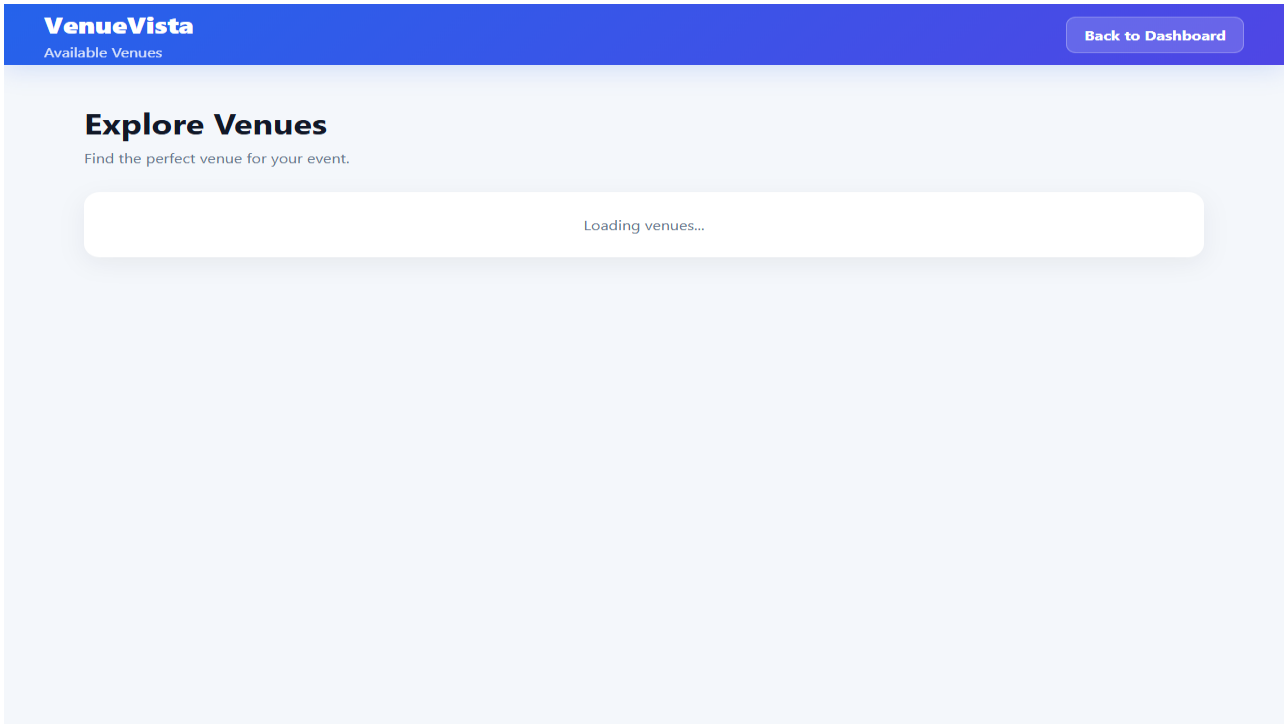



**Figure 4: Venue Discovery & Reservation Catalog (/venues)**

*Dynamic venue cards displaying location, capacity, daily rates, photos, and date selection booking modal.*


VenueVista
Available Venues
Back to Dashboard
Explore Venues
Find the perfect venue for your event.
Loading venues...

# Customer Ledger & Event Ticketing

**Figure 5: Customer 'My Bookings' Ledger (/my-bookings)**

*Displays real-time reservation states (Approved, Pending, and Rejected with host explanation reason box).*

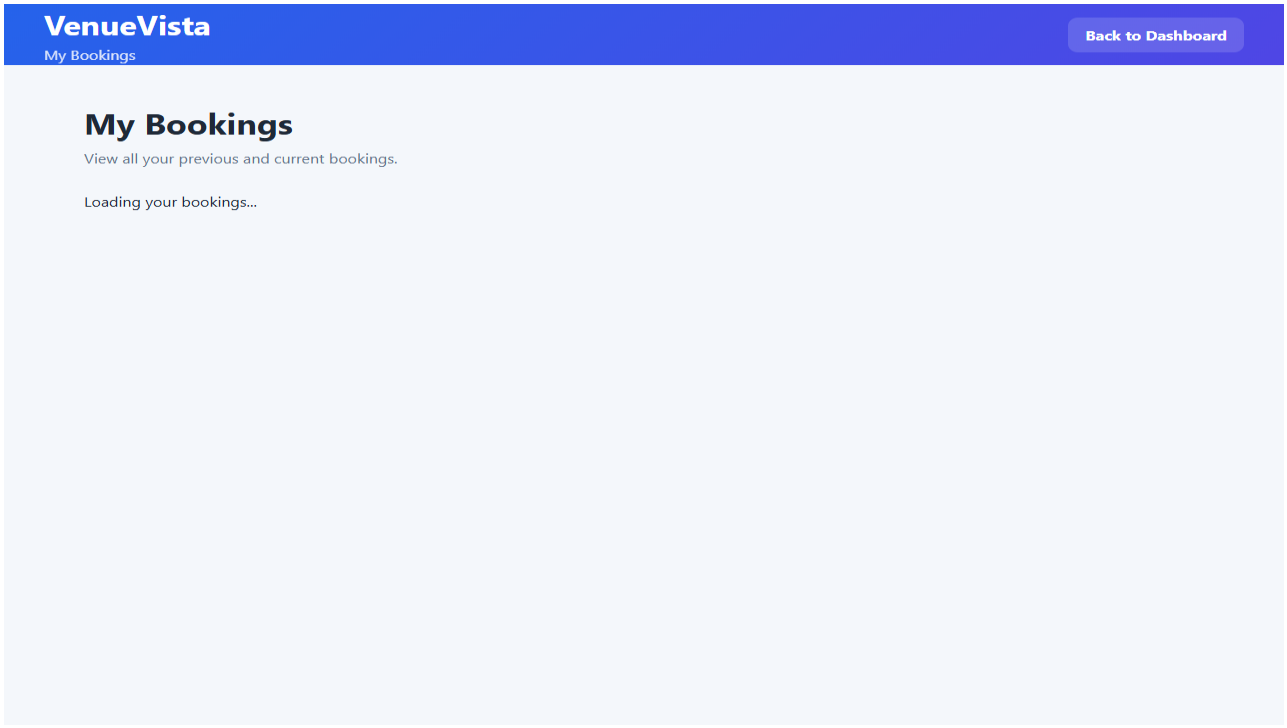
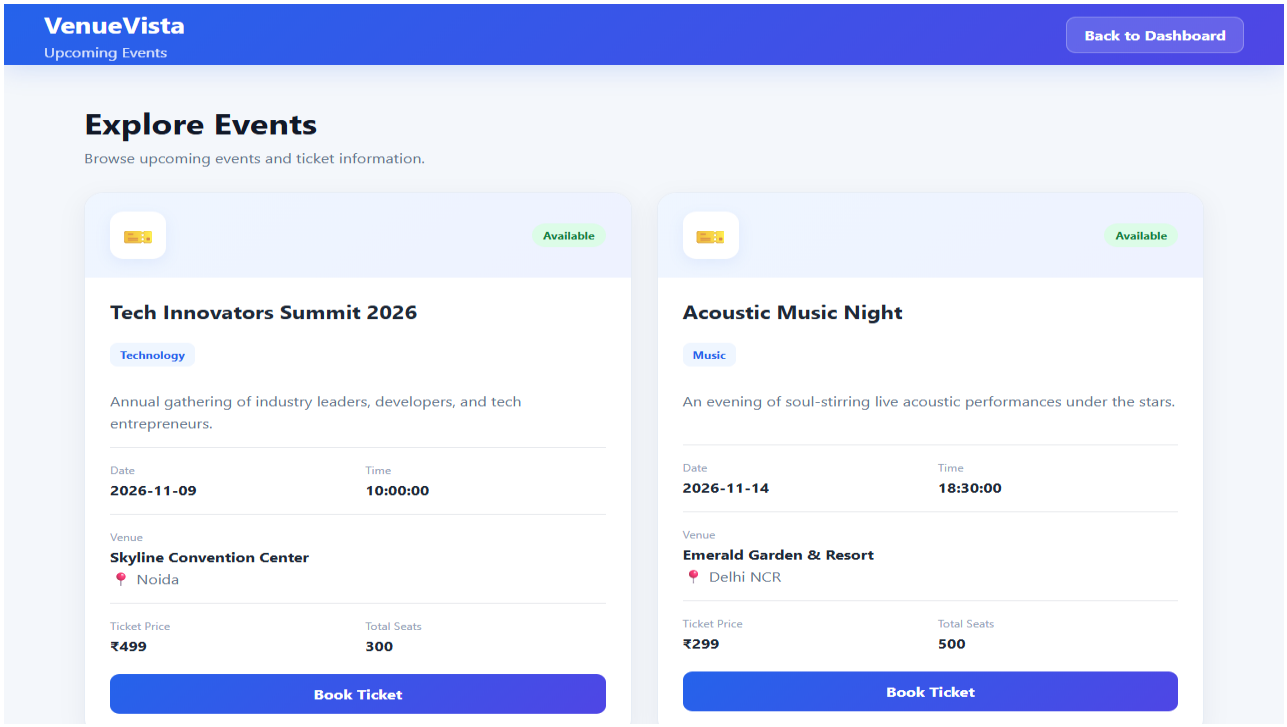



**Figure 6: Events & Ticket Reservation (/events)**

*Browse upcoming tech conferences, acoustic music nights, and concerts with seat booking integration.*


VenueVista
Upcoming Events
Back to Dashboard
Explore Events
Browse upcoming events and ticket information.
Available
Tech Innovators Summit 2026
Technology
Annual gathering of industry leaders, developers, and tech entrepreneurs.
Date
2026-11-09
Time
10:00:00
Venue
Skyline Convention Center
Noida
Ticket Price
₹499
Total Seats
300
Book Ticket
Available
Acoustic Music Night
Music
An evening of soul-stirring live acoustic performances under the stars.
Date
2026-11-14
Time
18:30:00
Venue
Emerald Garden & Resort
Delhi NCR
Ticket Price
₹299
Total Seats
500
Book Ticket

# Venue Host & Provider Onboarding

**Figure 7: Dedicated Venue Provider Portal (/provider/login)**

*Executive themed sign-in portal with 1-click 'Autofill Credentials' button and customer toggle link.*

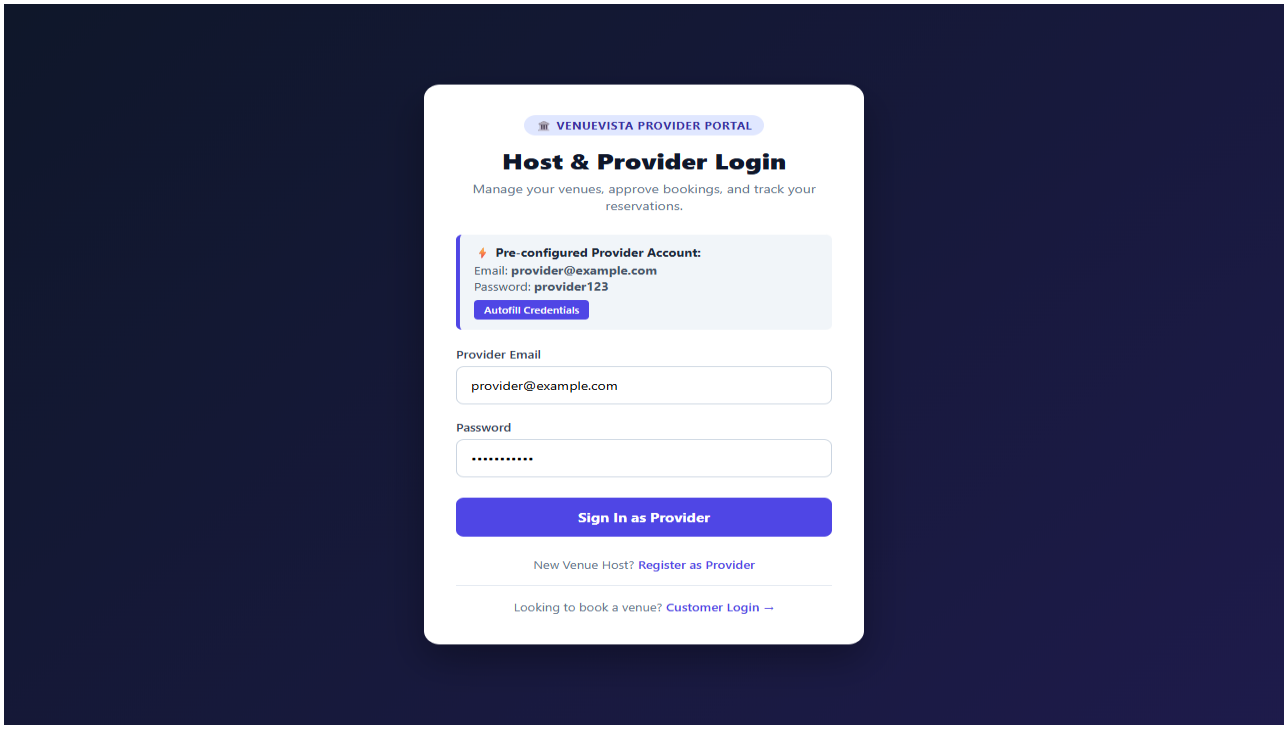
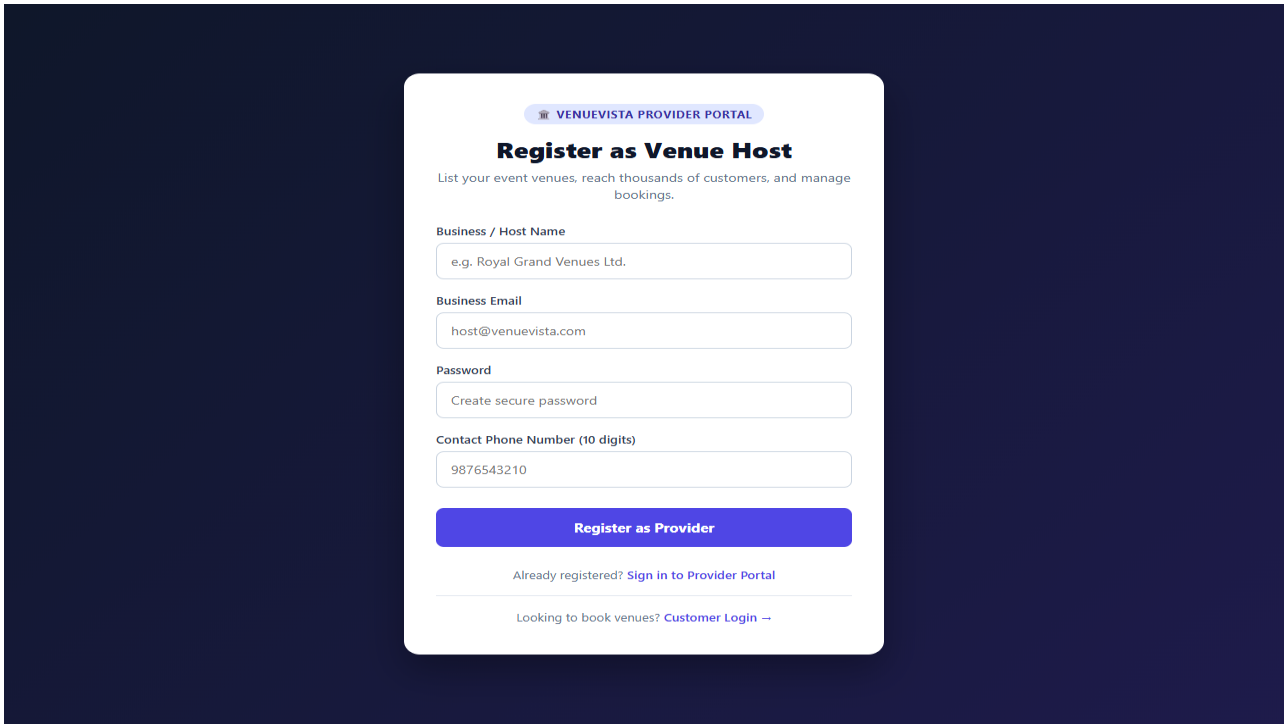



**Figure 8: Provider & Host Registration (/provider/register)**

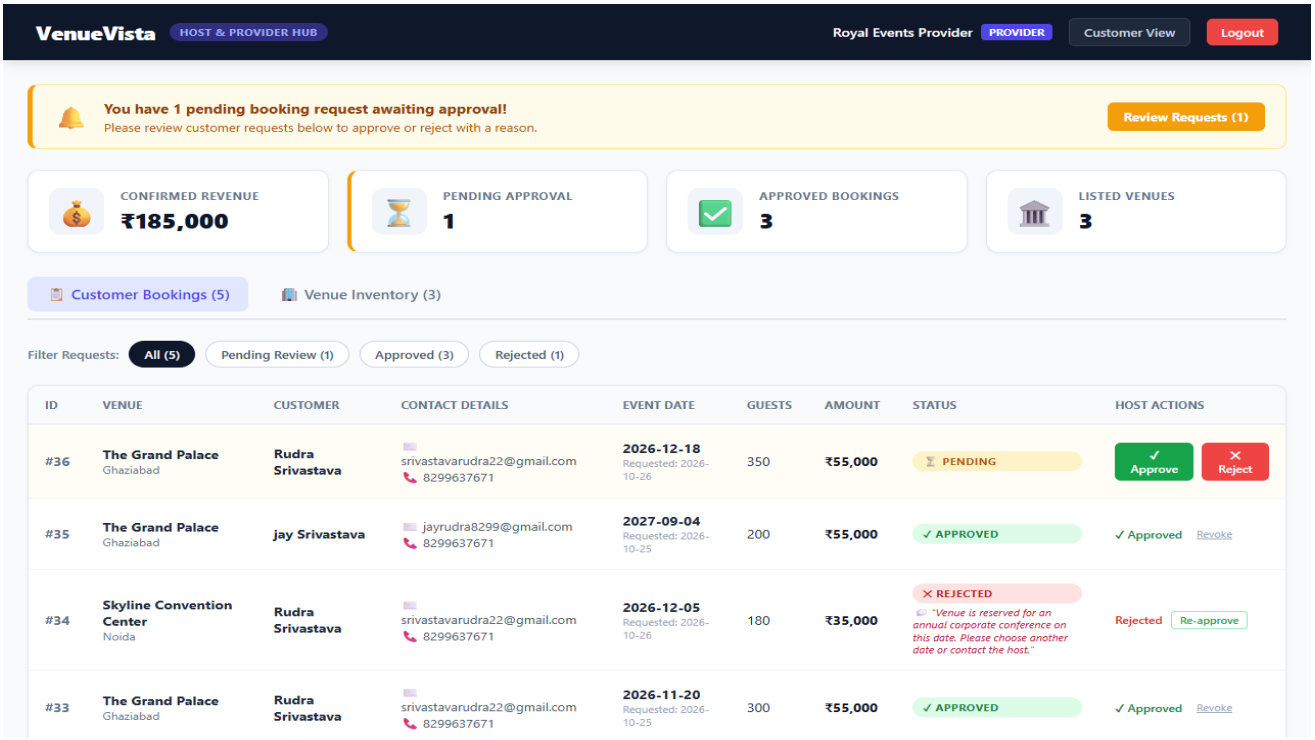
*Allows venue owners and event companies to register commercial host accounts with the PROVIDER role.*


VENUEVISTA PROVIDER PORTAL
Register as Venue Host
List your event venues, reach thousands of customers, and manage bookings.
Business / Host Name
e.g. Royal Grand Venues Ltd.
Business Email
host@venuevista.com
Password
Create secure password
Contact Phone Number (10 digits)
9876543210
Register as Provider
Already registered? Sign in to Provider Portal
Looking to book venues? Customer Login →

# Provider Business Management & Review Hub

**Figure 9: Provider Executive Dashboard (/provider/dashboard)**

*Real-time revenue metrics, pending review alerts, filterable reservations table, and approve/reject actions.*




**Key Capabilities of the Provider Dashboard:**

• **Live Financial KPIs:** Real-time aggregation of total confirmed revenue (■), total reservations, and active venues.

• **Pending Attention Banner:** Prominently alerts the host whenever new customer requests are awaiting review.

• **Customer Ledger Table:** Displays customer name, phone, email, requested event date, guest count, and total amount.

• **Interactive Decision Actions:** One-click ✓ Approve or ✕ Reject with required justification reason dialog.

• **Venue Inventory Management:** Add new properties with photo URL, capacity, and pricing, or delete existing venue listings.

# 5. Security, Verification & Conclusion

## Security Architecture & Hardening:

- **Cryptographic Password Hashing:** Passwords are encrypted using BCrypt prior to persistence, ensuring credentials cannot be retrieved even in raw database dumps.
- **Stateless JWT Authentication:** Tokens are signed with HMAC-SHA256, carrying user role claims to enable stateless session verification.
- **Client-Side Route Protection:** Angular Functional Guards (authGuard and providerGuard) restrict access to protected portals based on active authentication and role.
- **Cross-Origin Resource Sharing (CORS):** Configured WebMvcConfigurer explicitly allows Angular frontend requests (http://localhost:4200) while blocking unauthorized domains.
- **Data Integrity Validations:** Jakarta Validation annotations (@NotBlank, @Email, @Min, @Pattern) guarantee sanitized inputs.

## Project Evaluation & Verification:

The system was subjected to comprehensive end-to-end testing across customer booking creation, host request approval, rejection with custom rationale, seat availability checks, and multi-user authentication. Automated seed data initializes upon startup to provide immediate evaluation without manual setup. Both the backend (Java 21 Spring Boot) and frontend (Angular 21) pass continuous integration checks successfully.

## Conclusion:

**VenueVista** successfully demonstrates a complete, reliable, and scalable venue management platform. By introducing the dual-portal provider workflow and transparent rejection reasoning, it solves the real-world friction of uncoordinated event bookings, establishing a modern standard for venue reservation marketplaces.

| | | |
|---|---|---|
| **Report Generated:** October 2026 | **Repository:** github.com/srivastavarudra22-prog/venue-vista | **Build Status:** Passed (CI/CD) |